

Speedathon

22 maggio 2026

Il problema

Sono stati forniti il codice sorgente in `particles.c` e il corrispondente `Makefile`. Il vostro compito è ottimizzare il programma in modo che esegua il più velocemente possibile sulla macchina disponibile, mantenendo la correttezza del risultato.

Il codice genera N particelle dotate di posizione, velocità e massa e calcola le interazioni gravitazionali tra di loro (ignorando la costante gravitazionale G) e aggiornando la posizione e la velocità. Questo avviene calcolando per ogni coppia di particelle i e j le forze agenti su i dovute a j . Questi valori sono poi accumulati in `fx`, `fy` e `fz` (i tre diversi componenti) in modo da avere la somma di tutte le forze agenti su i dopo che si è iterato su tutte le particelle. Successivamente vengono modificate velocità e posizione delle particelle (qui viene utilizzato un intervallo temporale DT per l'aggiornamento).

Il codice implementa quindi un algoritmo con complessità $O(N^2)$. Sebbene l'algoritmo sia da mantenere come struttura, lo scopo di questa speedathon è migliorarne l'implementazione.

Task

Potete modificare sia `particles.c` sia il `Makefile`, purché il significato del calcolo resti invariato. L'obiettivo è ottenere la migliore prestazione possibile, intervenendo sia sul codice sia sulle opzioni di compilazione.

Sono ammesse, tra le altre, le seguenti tecniche:

- uso di OpenMP sulla CPU per eseguire in parallelo le parti adatte del codice;
- OpenMP con `target offloading`, cioè l'esecuzione di parti del calcolo su un acceleratore o su una GPU quando il compilatore e il sistema lo supportano;
- uso di OpenMP `simd` per sfruttare le istruzioni vettoriali del processore;
- uso del flag di compilazione `-ffast-math`.

Nel caso di un ciclo semplice come il seguente:

```
for (int i = 0; i < n; ++i) {  
    c[i] = a[i] + b[i];  
}
```

una direttiva come `#pragma omp simd` indica al compilatore che le iterazioni sono indipendenti e che quindi possono essere eseguite con istruzioni vettoriali. In pratica, invece di elaborare un solo elemento alla volta, il processore può

usare istruzioni vettoriali per elaborarne più di uno con una singola istruzione. Attenzione che state indicando che il loop può essere correttamente parallelizzato con istruzioni vettoriali!

Il flag `-ffast-math` abilita ottimizzazioni floating-point più aggressive. Questo può migliorare le prestazioni perché consente al compilatore di riordinare, combinare o semplificare alcune operazioni in virgola mobile, ma può anche introdurre piccole differenze numeriche dato che le operazioni nei floating point nello standard IEEE 754 non sono, per esempio, associative.

È inoltre consentito modificare le strutture dati utilizzate per rappresentare le particelle (i.e., non si è vincolati a usare la struttura in `particles.c`).

Specifiche

Il codice verrà eseguito su una macchina con 30 core, 200GB di memoria RAM e una GPU NVIDIA A10 con 24GB di VRAM con installato ubuntu 24.04 on sia `gcc` (versione 13.3) che `nvc` (versione 26.3) presenti.

Consegna

Il codice baseline è disponibile sul team del corso e anche all'URL: <https://drive.google.com/file/d/1rVwEp1a7Hp0dUe69vaKXU1St26F-X3lK/view?usp=sharing>

Url per la submission (copiate e incollate il testo del makefile e del sorgente c): <https://forms.gle/gs47wnBWg9QVxwwq8>